



1st Semester Exam Correction, 2024/2025

Exercise 1

(06 marks)

Consider the following program:

☐ &, int, ≤, 1, index (01.5pt)

☐ int X; X in lowercase : int x; . (0.5pt)

☐ the trace of this program for x=8. (0.5pt)

i	T[i]	index	found
/	/	-1	0
0	5	-1	0
1	2	-1	0
2	4	-1	0
3	8	3	1

☐ linear search in an array (0.5pt)

☐ b and d

☐ Rewrite the code using the while loop (01 marks)

```
1  #include <stdio.h>
2  int main(){
3  int found = 0;
4  const int N=5;
5  int X;
6  int T[]={5,2,4,8,9};
7  int index = -1;
8  scanf("%d", &x);
9  int i=0;
10 while (i<N){
11     if (T[i] == x) {
12         found = 1;
13         index =i;
14         break;
15     }
16     i++;
17 }
18 printf("the result is %d at
        index %d",found, index);
19 return 0;}
```

☐ Rewrite the code without using the keyword break (01 marks)

```
1  #include <stdio.h>
2  int main(){
3  int found = 0;
4  const int N=5;
5  int X;
6  int T[]={5,2,4,8,9};
7  int index = -1;
8  scanf("%d", &x);
9  int i=0;
10 while ((i<N)&&(!found)){
11     if (T[i] == x) {
12         found = 1;
13         index =i;
14     }
15     i++;
16 }
```

```

17     printf("the result is %d at
        index %d",found, index);

```

```

18     return 0;}

```

Exercise 2 (08 marks)

```

1  #include <stdio.h>
2  int main() {
3      int M=8;
4      int W[] = {7,5,3,2,4,6,1,3};
5      // int W[] = {4,6,7,4,5,3,2,4,1,6};
6      // Black zone
7      int black_area = 0;
8      if (M > 0) {
9          for (int i = 0; i < M; i++) {
10             //0.25
11             black_area += W[i]; //0.5
12         }
13     } // White zone 02 marks
14     int maX;
15     if (M <= 0) {
16         maX+ 0;
17     } else
18     {
19         maX = W[0];
20         for(int i = 1; i < M; i++)
21         {
22             if(W[i] > maX)
23             {
24                 maX = W[i];
25             }
26         }
27     }
28     int white_area = 0;
29     if (M > 0) {
30
31         for (int i = 0; i < M; i++) {
32             white_area += maX - W[i];
33         }
34     }
35     // Peaks number

```

```

36     int num_descends = 0;
37     if (M > 0) {
38         for (int i = 0; i < M-1; i++) {
39             if (W[i] > W[i + 1]) {
40                 num_descends++;
41             }
42         }
43     }
44     //Large rectangle area 03 marks
45     int large_rect = 0;
46     // Consider every bar one by one
47     for (int i = 0; i < M; i++) {
48         int c = W[i];
49         // Traverse left while we have a
50         // greater height bar
51         for (int j = i - 1; j >= 0 &&
52             W[j] >= W[i]; j--) {
53             c += W[i];
54         }
55         // Traverse right while we have
56         // a greater height bar
57         for (int j = i + 1; j < M &&
58             W[j] >= W[i]; j++) {
59             c += W[i];
60         }
61         if (c > large_rect) {
62             large_rect = c;
63         }
64     }
65     printf("H1: Black Area: %d, White
        Area: %d, Peaks: %d\n, Large
        rectangle: %d\n",
66         black_area, white_area,
67         num_descends,large_rect); //
68     return 0;
69 }

```

All solutions that work only for the given examples are incorrect.

Exercise 3 (06 marks)

```
1 #include <stdio.h>
2 int main()
3 {
4     int p=7;
5     int L[] = { 1, 2, 3, 5, 6,7,8 };
6     for (int i = 0; i < p; i++) {
7         T[i] = 1;
8     }
9     for (int i = 1; i < p; i++) {
10         for (int j = 0; j < i; j++) {
11             if (L[i] > L[j] && T[i] <
12                 T[j] + 1) {
13                 T[i] = T[j] + 1;
14             }
15         }
16         int maxT = 0;
17         for (int i = 0; i < p; i++) {
18             if (T[i] > maxT) {
19                 maxT = T[i];
20             }
21         }
22         // Return min changes for array to
23         // strictly increasing
24         printf("%d", p - maxT);
25     }
```

The following solution is not correct, it does not give the always the minimum changes because the code counts each decrease individually. in the given input the code gives minchanges = 2 so the array will be [1, 5, 6, 6, 7] but it is not strictly in increasing order.

```
1 #include <stdio.h>
2 int main() {
3     int L[] = {1, 5, 2, 6, 3};
4     int p =5;
5
6     if (p <= 0) {
7         printf("Array is empty.\n");
8         return 0;
9     }
10     int changes = 0;
11     for (int i = 1; i < p; i++) {
12         if (L[i] <= L[i - 1]) {
13             changes++;
14         }
15     }
16     printf("Minimum changes: %d\n",
17           changes);
18     return 0;
19 }
```